individual users, so stronger protections may be required, because the damage from compromise of a service account can be greater than any individual user account. In this chapter you'll learn how to authenticate services and additional hardening that can be applied to better protect privileged accounts, using advanced features of OAuth2.

> **NOTE** The examples in this chapter require a running Kubernetes installation configured according to the instructions in chapter 10.

11.1 API keys and JWT bearer authentication

One of the most common forms of service authentication is an *API key*, which is a simple bearer token that identifies the service client. An API key is very similar to the tokens you've used for user authentication in previous chapters, except that an API key identifies a service or business rather than a user and usually has a long expiry time. Typically, a user logs in to a developer portal and generates an API key, which they can then add to their production environment to authenticate API calls, as shown in figure 11.1.